

User Guide: EShop API & Contract Testing

Nhóm thực hiện: Group 05

Lưu ý cho nhóm: Đây là file User Guide dùng chung (bản hoàn chỉnh nộp cho giảng viên). Các thành viên hãy copy/viết phần nội dung do mình phụ trách vào các mục tương ứng dưới đây.

1. Giới thiệu (Introduction)

(Cả nhóm cùng viết) Tài liệu này hướng dẫn chi tiết cách thiết lập, chạy thử nghiệm và kiểm thử tự động hệ thống backend EShop bằng các công cụ Postman (API Testing), AI Test Generation và Pact (Contract Testing) thông qua CI/CD Pipeline.

2. Yêu cầu hệ thống và Cài đặt (Prerequisites)

- **Node.js & npm:** Phiên bản 16.x trở lên.
- **Postman:** Dùng để import và chạy API tests.
- **Newman CLI:** Dùng để chạy Postman trong CI/CD.
- **Git:** Dùng để pull mã nguồn.

3. Hướng dẫn chạy Hệ thống Backend (SUT)

1. Mở terminal và chuyển hướng vào thư mục `eshop-sut/backend`.
 2. Chạy lệnh: `npm install`
 3. Chạy lệnh: `node server.js` (hoặc `npm start`).
 4. Hệ thống sẽ lắng nghe ở địa chỉ: `http://localhost:3000`.
-

4. Hướng dẫn kiểm thử bằng Postman (API Testing)

(Phụ trách: Phạm Đức Toàn)

1. Cài đặt và Thiết lập cơ bản

1.1. Cài đặt Postman

- Truy cập [trang chủ Postman](#) và tải phiên bản phù hợp với hệ điều hành của bạn.
- Cài đặt và đăng nhập vào tài khoản Postman (hoặc sử dụng không cần tài khoản bằng cách click "Skip and go to the app").

1.2. Import Collection và Environment

- Mở Postman, chọn **Import** ở góc trái phía trên.
- Chọn file `EShop_Collection_v2.json` và `EShop_Environment.json` từ thư mục `Group05_06/Toàn/`.
- Sau khi import thành công, ở thanh bên trái (Sidebar) sẽ xuất hiện collection **EShop Collection v2**.
- Ở góc phải phía trên, click vào dropdown chọn Environment và đổi thành **EShop_Environment** (môi trường này chứa biến `{{baseUrl}}` trỏ đến `http://localhost:3000`).

2. Cấu trúc của Collection

Collection **EShop Collection v2** được tổ chức thành 4 thư mục chính, phản ánh các chức năng cốt lõi của EShop:

- **Authentication:** Chứa các API liên quan đến đăng nhập.
- **Products:** Chứa API lấy danh sách và chi tiết sản phẩm.
- **Cart:** Chứa API thêm sản phẩm vào giỏ hàng.
- **Checkout:** Chứa API đặt hàng và thanh toán.

Mỗi request bên trong đã được lập trình sẵn kịch bản kiểm thử (test scripts) trong tab **Tests** để tự động kiểm tra Status Code, Schema, và Logic dữ liệu trả về (cả trường hợp đúng - Happy Path, và trường hợp sai/cố tình bắt lỗi - Negative/Edge Path).

3. Cách chạy kiểm thử (Thực thi Test)

3.1. Chạy một Request đơn lẻ

1. Mở thư mục chứa API bạn muốn test (VD: **Authentication**).
2. Click chọn request **1.1 Happy Path – Đăng nhập thành công**.
3. Nhấn nút **Send** màu xanh ở góc phải.
4. Xem kết quả ở phía dưới màn hình:
 - Tab **Body:** Hiển thị dữ liệu API trả về (JSON).
 - Tab **Test Results:** Hiển thị kết quả tự động kiểm tra (VD: **PASS: Status code is 200**).

3.2. Chạy tự động toàn bộ Collection (Collection Runner)

Thay vì chạy từng API thủ công, bạn có thể chạy toàn bộ kịch bản kiểm thử bằng một click:

1. Click vào biểu tượng dấu 3 chấm **...** bên cạnh tên collection **EShop Collection v2**.
 2. Chọn **Run collection**.
 3. Màn hình Runner sẽ hiện ra, liệt kê toàn bộ các requests. Bạn có thể giữ nguyên cài đặt mặc định (Iterations: 1, Delay: 0).
 4. Nhấn nút **Run EShop Collection v2**.
 5. Đọc bảng tóm tắt kết quả (Run Summary): Postman sẽ báo có bao nhiêu test passed (xanh) và bao nhiêu test failed (đỏ).
 - Những test failed là do API backend có lỗi (thiếu validation dữ liệu), không vượt qua được các kịch bản kiểm tra nghiêm ngặt của bộ test.
-

5. Hướng dẫn sinh kịch bản Test bằng AI

(Phụ trách: **Nguyễn Quang Đăng Khoa**)

Mục này cung cấp hướng dẫn chuyên sâu về việc tích hợp các mô hình AI (ChatGPT, Gemini, Claude) vào quy trình kiểm thử API tự động. Mục tiêu không phải là để AI thay thế QA, mà sử dụng AI như một công cụ tối ưu

hóa tốc độ khởi tạo (Boilerplate Generator), kết hợp với quy trình kiểm duyệt chặt chẽ nhằm tối đa hóa hiệu suất CI/CD.

5.1. Quy trình sinh kịch bản tiêu chuẩn (Human-in-the-loop)

Để AI có thể sinh ra các script Postman chất lượng cao, đúng định dạng và có tính liên kết, kỹ sư kiểm thử cần tuân thủ 4 bước sau:

Bước 1: Chuẩn bị Ngữ cảnh

- Tuyệt đối không yêu cầu AI viết test bằng các câu lệnh chung chung.
- Cần cung cấp toàn bộ nội dung của **Tài liệu Đặc tả API (API Specification)** cho AI. Điều này giúp AI hiểu rõ các Endpoint, cấu trúc Body (JSON), Header yêu cầu (ví dụ: **Authorization: Bearer <token>**) và các mã lỗi HTTP dự kiến.

Bước 2: Sử dụng Prompt Engineering

- Sử dụng các mẫu câu lệnh đã được chuẩn hóa trong file thư viện **prompt_library.md**.
- Phân chia nhỏ yêu cầu: Thay vì bắt AI viết toàn bộ hệ thống cùng lúc, hãy yêu cầu theo từng cụm nghiệp vụ. Ví dụ: Cụm Authentication (**/api/login**, **/api/register**), cụm Order Lifecycle (**/api/cart** -> **/api/checkout** -> **/api/orders/:id**).

Bước 3: Trích xuất và Tích hợp

- Yêu cầu AI xuất kết quả dưới dạng chuỗi JSON tuân thủ chuẩn **Postman Collection Schema v2.1.0**.
- Sao chép khối JSON, lưu thành file **.json** và sử dụng tính năng **Import** của Postman để đưa vào Workspace mà không cần thiết lập thủ công.

Bước 4: Review & Refactor

- Kỹ sư QA trực tiếp kiểm tra lại các kịch bản. AI thường làm tốt phần thiết lập API skeleton (URL, Headers, Payload), nhưng phần Script Validation (**pm.test**) và các rủi ro bảo mật cần được kỹ sư trực tiếp bổ sung và tinh chỉnh lại.

5.2. Nhận diện và Khắc phục các "Điểm mù" của AI (AI Failure Modes)

Trong quá trình sinh code, AI thường mắc phải các lỗi luận lý ngầm. Việc không nhận diện được các "điểm mù" này sẽ dẫn đến hiện tượng **False Positive** (kiểm thử báo Pass nhưng thực tế hệ thống có lỗi). Khi sử dụng AI, QA Engineer đặc biệt phải rà soát các trường hợp sau:

1. Lỗi Schema Validation rộng (False Positives)

- Hiện tượng:** AI thường sinh các đoạn code kiểm tra cấu trúc JSON (Schema) rất hời hợt. Nó thường khởi tạo mảng **"properties": {}** và đặt **"additionalProperties": true**.
- Hậu quả:** Postman sẽ chấp nhận mọi dữ liệu trả về từ Server. Dù API trả về object rỗng **{}** hay chuỗi HTML chứa lỗi nội bộ (Internal Server Error), Test Case vẫn hiện màu xanh (Pass).
- Biện pháp:** Qua prompt, ép AI phải khai báo chi tiết các trường bắt buộc (**required**), định nghĩa rõ ràng Data Type (String, Integer) và thiết lập **"additionalProperties": false** để chặn lỗi Data Leakage.

2. Bỏ qua Ràng buộc Nghiệp vụ (Data Hallucination)

- **Hiện tượng:** AI sinh ra payload đúng cú pháp JSON chuẩn nhưng bỏ qua hoàn toàn các ràng buộc logic kinh doanh (Business Rules).
- **Ví dụ thực tế:** Đối với API áp dụng mã giảm giá (`POST /api/apply-coupon`), AI có thể tạo request gửi mã thành công nhưng hoàn toàn bỏ qua việc viết test case kiểm tra điều kiện `min_order_amount` (giá trị đơn hàng tối thiểu) hoặc `max_uses_per_user` (số lần dùng tối đa).
- **Biện pháp:** Kỹ sư QA phải tự thiết kế và bổ sung các "Manual Edge Cases" để kiểm thử Boundary values và các luồng thất bại.

3. Lỗi hỏng Tư duy Phân quyền (RBAC Failure)

- **Hiện tượng:** AI mặc định sử dụng chung một biến `{{token}}` cho mọi request và ngầm định rằng mọi API đều được gọi ở trạng thái ủy quyền cao nhất (Happy-path).
- **Ví dụ thực tế:** AI gần như sẽ bỏ sót việc thiết kế test case dùng tài khoản User thông thường để gọi thử API dành riêng cho Admin, chẳng hạn như API xóa người dùng (`DELETE /api/admin/users/:id`) hoặc lấy danh sách toàn bộ đơn hàng (`GET /api/admin/orders`).
- **Biện pháp:** Yêu cầu AI tạo riêng một Folder "Security Tests", chỉ định viết các kịch bản: Broken Access Control, Privilege Escalation (Leo thang đặc quyền), và IDOR.

4. Thiên kiến Happy-path và Dữ liệu tĩnh (Static Data)

- **Hiện tượng:** AI thích viết các Test Case thành công và thường hardcoded (gắn cứng) dữ liệu tĩnh (Ví dụ: `email: "test@domain.com"`).
- **Hậu quả:** Khi đưa Collection vào chạy tự động trên CI/CD Pipeline (như GitHub Actions), lần chạy thứ 2 sẽ lập tức báo lỗi Conflict (HTTP 409) do trùng lặp dữ liệu trong Database.
- **Biện pháp:** Bắt buộc AI sử dụng các biến động (Dynamic Data) của Postman như `{{randomEmail}}`, `{{randomUUID}}` trong tab Pre-request Script.

6. Hướng dẫn kiểm thử bằng Pact (Contract Testing)

(Phụ trách: Nguyễn Nhật Nam)

Tài liệu này hướng dẫn chi tiết cách chạy Consumer Test để sinh file hợp đồng (Pact file) và Provider Verification để kiểm chứng hợp đồng đối với hệ thống EShop.

1. Consumer Test (Tạo Hợp Đồng)

Nhiệm vụ của Consumer (Client) là định nghĩa các kỳ vọng (expectations) đối với API và sinh ra bản hợp đồng dưới dạng file JSON.

Các bước thực hiện:

1. **Khởi tạo cấu hình Pact:** Sử dụng `PactV3` để khai báo tên `consumer` và `provider`.
2. **Định nghĩa Interactions:**
 - `.given(...)`: Thiết lập trạng thái giả định (VD: "Giỏ hàng rỗng").
 - `.uponReceiving(...)`: Đặt tên kịch bản kiểm thử.
 - `.withRequest(...)`: Cấu hình request (Method, Path, Headers).
 - `.willRespondWith(...)`: Định nghĩa response mong đợi. **Lưu ý:** Luôn sử dụng các Matchers (như `MatchersV3.like`) thay vì gán giá trị cứng để tránh Flaky test.
3. **Thực thi Test:** Gửi request thực tế thông qua thư viện Axios hoặc Fetch tới Mock Server của Pact.

4. **Kết quả:** Nếu test Pass, Pact sẽ tự động sinh/cập nhật file JSON contract vào thư mục `pacts/`.

Lệnh chạy Consumer Test:

```
npx jest Group05_07/Nam/test/*.consumer.test.js
```

2. Provider Verification (Xác Minh Hợp Đồng)

Nhiệm vụ của Provider (Backend) là khởi chạy server API thực tế và xác minh lại các yêu cầu từ file Contract đã được Consumer tạo ra.

Các bước thực hiện:

1. **Chuẩn bị Server:** Khởi động backend server trên một port test (VD: 3000) cùng database chuyên dụng cho test.
2. **Cấu hình Verifier:** Khởi tạo `Verifier` từ thư viện `@pact-foundation/pact`.
3. **Thiết lập thông số Verification:**
 - `providerBaseUrl`: Trỏ tới server API đang chạy.
 - `pactUrls`: Trỏ tới file JSON contract.
 - `stateHandlers` (**Quan trọng nhất**): Viết các hàm logic setup/teardown dữ liệu khớp chính xác 100% với chuỗi trạng thái (`given`) mà Consumer đã định nghĩa.
4. **Thực thi Xác minh:** Gọi hàm `verifyProvider()`. Pact sẽ đọc file contract, tự động gọi state handler tương ứng, gửi request vào server thật và so sánh kết quả.

Lệnh chạy Provider Test:

```
npx jest Group05_07/Nam/test/provider.test.js
```

7. Hướng dẫn tự động hóa (CI/CD Pipeline)

(Phụ trách: Huỳnh Sĩ Luân)

Hệ thống CI/CD của EShop được triển khai qua **GitHub Actions** nhằm tự động hóa quá trình chạy kiểm thử API và xác thực hợp đồng Pact mỗi khi mã nguồn thay đổi.

7.1. Cách kích hoạt Pipeline (Trigger Pipeline)

Pipeline được thiết lập tự động chạy trong hai trường hợp chính:

1. **Push:** Khi có bất kỳ commit nào được đẩy trực tiếp lên nhánh `main`.
2. **Pull Request:** Khi một thành viên tạo Pull Request (PR) yêu cầu gộp code từ nhánh tính năng (nhánh phụ) vào nhánh `main`. Quy trình này hoạt động như một chốt chặn bảo vệ.

7.2. Cách theo dõi tiến trình chạy trên GitHub Actions

1. Truy cập trang GitHub của dự án EShop.

2. Nhấp chọn tab **Actions** trên menu thanh công cụ chính.
3. Chọn run mới nhất ở danh sách phía dưới (tên run trùng với tên commit).
4. Click chọn job **test-api** ở cột bên trái để chuyển vào giao diện xem log.
5. Tại đây, bạn có thể quan sát trực tiếp kết quả chạy từng bước: cài đặt môi trường, chạy Pact Consumer, khởi động server backend, quét Newman API Test v2, và chạy Pact Provider Verification.

7.3. Cách tải và xem kết quả báo cáo (Newman & Pact Logs)

Quy trình tự động gom toàn bộ các tệp báo cáo sinh ra trong quá trình kiểm thử vào một thư mục **reports/** để đẩy lên thành Artifact.

1. Ở trang tóm tắt lần chạy của Actions (Run Summary), cuộn xuống cuối cùng tìm phần **Artifacts**.
2. Click vào tên **EShop-Test-Reports** để tải về tệp tin nén dạng **.zip**.
3. Giải nén tệp tin này, bạn sẽ nhận được 2 tệp báo cáo chính:
 - **newman-report.html (Báo cáo Newman)**: Mở tệp tin này bằng bất kỳ trình duyệt web nào (Chrome, Firefox, Edge). Giao diện trực quan sẽ hiển thị toàn bộ kết quả kiểm thử API v2, bao gồm: tỷ lệ pass/fail của các assertion, chi tiết gói tin request/response và thời gian phản hồi của từng endpoint API.
 - **pact-verification-report.txt (Log xác minh Pact)**: Mở bằng text editor. Tệp tin ghi nhận kết quả so khớp hợp đồng của Pact Provider. Nếu backend làm sai cam kết hợp đồng với Consumer, chi tiết lỗi mismatch (như sai kiểu dữ liệu hay thiếu key dữ liệu) sẽ được ghi cụ thể tại đây.

8. Các "Điểm mù" của công cụ (Failure Modes)

(Lưu ý từ Rubric: Mục này phải tổng hợp Failure Mode của tất cả các công cụ vào chung một nơi, KHÔNG ĐƯỢC tách rời rạc từng phần).

8.1. Postman Failure Modes

- **Lỗi so sánh kiểu dữ liệu ngầm định (Type Mismatch)**: Khi API backend trả về dữ liệu bị sai kiểu (ví dụ trả về **price** dạng chuỗi **"28000000"** thay vì số nguyên **28000000**), nếu trong tab Tests bạn dùng **pm.expect(price).to.eql(28000000);**, test sẽ báo fail với thông báo mơ hồ như **AssertionError: expected '28000000' to deeply equal 28000000**. Người mới dùng rất dễ nhầm lẫn rằng logic API tính toán sai giá tiền, chứ không nhận ra đó là lỗi ép kiểu dữ liệu.
- **Quên cấu hình Environment/Token (Unresolved Variables)**: Nếu bạn quên chọn Environment (đang ở trạng thái "No Environment"), các biến như **{{baseUrl}}** hay **{{token}}** sẽ không có giá trị. Tuy nhiên, Postman **không báo lỗi ngay** mà gửi nguyên cái chuỗi chữ **{{baseUrl}}** đi làm URL. Kết quả là hệ thống trả về lỗi **404 Not Found** hoặc **401 Unauthorized**. Người test sẽ mất nhiều thời gian debug API trong khi nguyên nhân chỉ là quên chọn cấu hình môi trường.
- **Lỗi "Silent Failure" trong script JavaScript (Bỏ qua lỗi ngầm)**: Tab **Tests** trong Postman sử dụng mã JavaScript. Nếu bạn lỡ tay viết sai cú pháp (syntax error) hoặc gọi một hàm không tồn tại ở nửa đầu đoạn script, đoạn script đó sẽ bị crash. Tuy nhiên, **Postman không báo lỗi cú pháp đỏ rực lên màn hình UI**, mà nó chỉ âm thầm skip (bỏ qua) các câu lệnh **pm.test** bên dưới. Hệ quả là số lượng Test Cases trong tab "Test Results" bị giảm đi, và nếu các test chạy được đều Pass, UI vẫn hiện màu xanh "Pass" đánh lừa người dùng rằng mọi thứ đều ổn định.

8.2. AI Generation Failure Modes

- **Schema Validation rỗng (False Positives nguy hiểm):** Khi yêu cầu sinh test, AI thường tạo JSON Schema rỗng (VD: `"properties": {}, "additionalProperties": true`). Hệ quả là Postman sẽ chấp nhận mọi dữ liệu trả về (kể cả khi hệ thống sập hoặc trả về thông báo lỗi), làm xuất hiện trạng thái "Pass" ảo giác vô cùng nguy hiểm.
- **Bỏ qua ràng buộc nghiệp vụ (Business Rules Ignorance):** AI chỉ có khả năng lắp ráp payload đúng định dạng (cấu trúc JSON) nhưng không hiểu logic nghiệp vụ sâu. Ví dụ: API yêu cầu đơn hàng phải trên 500k mới được áp mã giảm giá, nhưng AI vẫn sinh test áp mã thành công cho đơn 100k, dẫn đến việc bỏ lọt lỗi nghiệp vụ.
- **Thiếu tư duy kiểm thử phân quyền (RBAC & IDOR Failure):** AI mặc định dùng duy nhất một biến `{{token}}` cho toàn bộ mọi request. Nó hoàn toàn bỏ sót các kịch bản kiểm thử bảo mật như: User thường gọi API của Admin, hoặc User A xem đơn hàng của User B (lỗi IDOR).
- **Thiên kiến Happy-path và dữ liệu tĩnh:** AI có xu hướng chỉ viết các kịch bản thành công (Happy-path) và gán cứng (hardcode) dữ liệu. Nếu chạy tự động trên CI/CD nhiều lần, dữ liệu tĩnh này sẽ gây xung đột (VD: tạo trùng email). AI cũng hiếm khi tự sinh các Boundary Value (giá trị âm, rỗng, null) để test Backend Validation.

8.3. Pact Failure Modes

- **Silent Schema Change (Postel's Law):** Khác với sự chặt chẽ của Postman hay sự "hời hợt" của AI, Pact chỉ kiểm tra đúng những gì Consumer yêu cầu. Nếu Backend thêm trường dữ liệu mới (làm thay đổi ngầm luồng nghiệp vụ), Pact vẫn Pass.
- **Phụ thuộc 100% vào Consumer Test:** Tương tự như "Thiên kiến Happy-path" của AI, nếu Consumer quên viết contract cho trường hợp báo lỗi (VD: 401 Unauthorized), thì việc Backend vô tình gỡ bỏ xác thực vẫn sẽ lọt qua mắt Pact.
- **Bùng nổ chi phí bảo trì (State Handlers):** Khi số lượng API tăng, việc duy trì trạng thái test (`stateHandlers`) phía Provider trở thành ác mộng. Nếu thiếu cơ chế teardown dọn dẹp data, dữ liệu rác sẽ gây xung đột, tạo ra "Flaky tests" (lỗi ngẫu nhiên), điều mà Postman hay AI ít gặp hơn trên môi trường test dùng một lần.
- **Bỏ qua Performance:** Dù API phản hồi chậm gấp 50 lần, Pact vẫn Pass miễn là schema đúng, trái ngược với Postman có thể set timeout cứng trong script.